audio1317915294

Transcribed by TurboScribe. Go Unlimited to remove this message.

Oh, yeah, thanks. Recording is now started. Okay, great.

Yeah. So what are the commands that we have seen last week? Can we have quickly have those in the chat box? Because once we start the session, I am generally I don't see chat box enough because I have only one screen. Yes, give me, give me add status, commit.

Yeah, that's the porcelain command. But what are the commands that we have seen to derive? Why do we need at all get in it in the first place? Right, so Vedant has answered. Hash object, yes.

What else? So hash object, we have your cat file, right? So Om Patel has answered cat file B, yes. So we have started with a what is an application, how it is very similar to an application. That's how we started with the grid.

And then we have detoured we have basically we have seen a normal grid commands we did first in it so that every one of us is familiar with the basic workflow of grid. And then we wanted to derive I am still remembering when the powerful statement I don't know the name, sorry for that. But someone has said that get required get in it for the state management.

That was a really powerful statement for that day. Right. So that is what impactful statement that is what key engineering statement that get in it is used for state management.

Yes, Siddharth, you are right. Yes. I remember the name actually.

So good. Right. So today we will go even deeper.

We'll understand things from first principles. And right now the next question, the agenda, the way we going to learn today is that basically questioning it. Prove me that you are certainly using SHA-1.

Okay. So we have did bit of hashing. We have seen four constraints of hashing.

Anybody knows it? By any chance, if you're taking notes, what are the constraints of hashing we have seen? Yes. Unidirectional. Correct.

Amir, what else? Collision we haven't seen yet. We will see that today. We have seen four constraints.

Yes. Slightest change in input will drastic change in output, which is avalanche effect. Yes.

What else? So right now we have two constraints, basically, let's say three, like always producing

a cache for the data. Correct. From output, we cannot derive.

Correct. Yes. I guess someone has, Divyansh has entered all four constraints.

So let's have a big clap for Divyansh. Okay. Let's that energy sink in.

Great. Great. Okay.

Then let's get started. Have you opened your git terminal, git bash terminal? Yes. Because hands-on is the only way.

Right. So I'm not going to do much talking. We'll have, I will share the screen and then I want you to follow along.

Okay. After sharing the screen, I will use some context switch in between. We'll ask you whether are we going good? Is the right pace or not? Okay.

So I will not have much, much attention to the chat box. Okay. Shall I know like what are the concepts that are covered in the entire DevOps course before moving on? Sure.

Okay. My bad. I will share the document of session.

So it's more on the git GitHub, enterprise GitHub practices, then Docker and then Kubernetes concepts, and then we will do a GitHub actions. So these are the end-to-end, I would say, topics that we are going to discuss because we have close to eight to 10 weeks. So I'm firstly focusing on the core engineering behind GitHub or other DevOps because that is the entry point for you.

That's when you push your code in your repo and that is the start of your pipeline. So for those who have some idea about GitHub, you know, right? So when you push, that's where your pipeline begins. And now pipeline doesn't matter which tool you're using.

It can be Jenkins. It can be AWS CodePipeline. It can be AWS DevOps.

It can be CircleCI, TravisCI or Adobe Actions or Jenkins. So that doesn't matter. It's a tool, right? So, but ultimately everyone, every tool has some kind of pipeline there built in.

So let's focus on the engineering part of it, concept sort of it, and then anyway, writing the YAML files, creating the YAML files or Docker file, you will do more better than me because you are quite takes away. I'm really, it's really hard for me to get into this co-pilot chat and all this, right? Okay. So let's get started team.

Let me share my screen now. Let me know if you can see my screen. Yes.

By the way, anyone knows why generally it starts with hello world. Anybody is old enough here or I'm the only one who's born in 80s. Oh yeah.

I'm going to share a repo and whatever topic we will discuss, I will create a AI generated image that will cover all the commands. Yes, for testing purposes. I guess Vedha, what's age you are? So Vedha is probably right.

So printf is used for debugging purpose. In that days, we don't have a kind of tools like, so in those days, we don't have tools like we have now, right? Where we can put breakpoint and we basically stop the execution, right? So with the printf, we used to do debugging. Great.

So good curiosity and good guess. Okay. So let's get started.

What we are going to cover is the first place, how those hashes are created, or maybe you can question get that, prove me those hashes are SHA. So what we can do is let's first, just a quick recap, echo, hello world. Now, you know, if you do any casing changes or any spacing changes, you will get different hash.

So I'm completely okay with it, right? hyphen hyphen stdin and hyphen w. Okay. So I will put this into the chat box. If you are exactly following me, then you should get this ID.

We have seen this, but just for the sake of completeness, this is where we have stopped. Yes, last week. Okay.

Right now, this is 40 characters, hexadecimal, we have discussed that as well. And then we also seen how this ID gets stored in the git database, right? So if you go to this users, get underscore PLC 03. And if I view the hidden atoms, that's where you see into the objects.

The first two characters are the folder and the rest is a file name. Okay. So we have seen this.

Now, let's, let's prove this, that this is a SHA1, right? How we can prove this? Now, it's totally depend on your SHA, git installation. Okay. So, if you can see this tool, okay, type this, and you should not see the cursor blinking, just type Ctrl C after that.

If you don't see any error, that means this tool is there, right? So basically, when you install git, this tool is available called Shasm. Okay. For Regisa, STD in, after STD in, you need to have a space.

You need to have a space after STD in, in case if you are getting that STD in W. So just give a space after STD in, right? So, great. So, just like the output from Joel, if you have that tool, Shasm, because then now we are going a bit deeper and we will basically ask git prove me that it's Shasm. Okay, great.

So I have got few responses. So we can start. So what we'll do exactly same thing.

We'll do echo, hello world, but this time we will not use git. Yeah, just put controls. Okay.

Those who don't have, try to get Shasm from internet or something, or some bundling tool. So you, you might get from open SSH bundling tools, or you can focus on screen and you can see. Okay.

So what we'll do is Shasm, and I will copy paste it over here. My bad. What I have missed in this, I have missed the exclamation mark.

Okay. So I missed exclamation mark. So I will again copy and paste.

Yeah. Those who have, tell me the output. Is it exactly matching with the git? Those who have Shasm.

Okay. Divash. No, no, no.

Let me see what is I get. So 60 FD. So Drishti, whatever Drishti has shared, we should like that because she has exactly followed my command.

I mean, you can use any text doesn't matter as long as we are on the same page. Right. So Drishti and Ohm.

Okay. Irfan as well. Great.

Right. So we have made a mistake. Is it, or git is wrong? What do you think? Did we make any mistake or git is wrong? Because if it's SHA1, it has to be same, right? If git is using SHA1, then it has to be same.

Correct? Yes or no guys. So what do you think? Did we some make mistake or git is wrong? Neither. Right.

So we need to figure out what is exactly happening. Both are not wrong because what we did is we have exactly passed hello world as is to git, right? But git is doing some additional work. So what is that work? Let's figure it out.

No worries. If I get some exe, I will share that into the git as well. So even if you don't get SHA1, it doesn't matter.

Just see the process, how we can derive that, whatever the id git is basically sharing with us. Okay. So I will first, I will, what I will do is I will share you one more command.

I mean, you already, we already see this git cat file hyphen p. That's what to retrieve our hello world from the terminal, right? We have seen this, but then there is one more command in cat file, which is hyphen p. Instead of hyphen p, what we do is hyphen t, which is type. What is the type of this? When you type this, you will have of type blob. Now, why, why we are discussing this? Because the way git is doing is we need to type blob, then space.

Then we need to basically count the number of characters in the string. So h, e, double l, o, r, and o is 5, 6, space, 7, 8, 9, 10, 11, 12, 13, right? 1, 2, 3, 4, 5, 6, space, 7, 8, 9, 10, 11, 12, 13, SHA sum. Okay.

So 13 letters. So blob, then space, 13, and this backslash n or backslash 0, specifically not n, backslash 0. I will tell what is happening. So just give me a minute.

And then what we need to do is we need to copy entire thing, which is the content list. And then we need to pass this to SHA sum. So the way, right? I mean, getting it is we need to do first.

That's what we have done last time, right? So it is required. Without getting it, you will not be able to proceed now, right? So, okay. So I'm going to copy paste this into the chat box, and then we will see this.

Okay. So what happened? The way git has created the mechanism for hashing is that you would gonna need the type, then the total size of your string, in our case 13, and then the exact content that you had. So what is the output now? Still not? Oh, come on, guys.

I don't believe you. Let me check myself. Still weird, right? 9e.

Still 9e. Okay. So and you know, right? I like to make mistakes.

Yes. So I am deliberately making mistakes for sure. Because there is some subtle point in the, what is backslash 0? Backslash 0 is a character termination, null terminating character is basically telling that after 13 bytes, end that string.

So whatever object we are passing, it is after. So after it should terminate, and that is the reason we need to mention. So this is a format of git.

Basically, it needs type, space, and then this entire string. Okay. So what we'll do is now, now, what is the challenge? Okay.

So look at the challenge here. So when you use git bash, specifically bash, what bash does is it adds additional newline character in your when you type echo command. Okay.

So what if I do, if I make it this 14? So why 14? So hello world, all characters, one bytes, and then additional newline character that is inserted by echo command or shell. Can you now provide me the output after? Again, if you're not exactly following this string, then in that case, you need to count your characters. Okay.

Oh, come on. I thought this time it will be it should work this time. Okay.

5AD, at least Prajna, Sarforaj, Mukesh, then Vedant. Okay. We are still, we're still together.

Okay. But we are still far from 8AB. Okay.

Remember, this time we are not utilizing git. We are completely utilizing Shasam. Okay.

Now let's go again. Now what I need to do is I need to tell git is that this backslash zero is a escape sequence. So consider it as a escape sequence.

So hyphen e. Yeah, sure. I will show the previous command. I will show the previous command.

No worries. Can you now try this? If not, then I have made some mistake for sure. Can you now try this? Oh, great.

So we got our ID back. Right. Are you happy now? So we got our ID back.

Right. So this is the format that git uses for hashing. Okay.

So what it does. So why, why 14? You know, right? Because you have added one, not you. So basically, shell have added one extra line character, and that becomes string size as 14.

And then we also need to tell shell that please consider backslash zero as escape sequence, not for your hashing. Are we good so far? Are we good so far? Yes. So I will quickly show all the commands.

I think you already know cat file. That's the reason I didn't copy pasted it, but all good. But your question should be like this.

Mohith, for hello world, it's okay to count the characters, right? For hello world, you are all good to count the number of characters. What if I have my YAML file or my agent? What if I have my agent Python file? Or what if I have bigger code base in just one simple Python file? How I can count the number of characters? In that case, with cat file, you have one more option called size. So hyphen s, it will get done.

So if you have hash key, just type cat file hyphen s, you will get the exact size of your blob. And that way, you will have 14. Are we good? So P is for pretty print.

So if last session, if you're not able to recalling guys, then recording is the I would say best place, right? So P is for pretty print just to basically extract the content from the from the files. Though I'm using word extract, but that's not exactly that's how it works. We will see that what, how and what happens when we write into it.

And then this is for checking the size of that content. Okay. I will again, just quickly explain what we have over here, right? This in this command, we haven't used git.

I hope you're with me, right? In this command, we have used Shasm, not the git. So Shasm is the tool which will convert your input into the SHA-1. So then there is a SHA-256 sum.

So there is a SHA-256 and then there is a SHA-512. So there are multiple tools, Shasm also

known as SHA-1 sum, right? So what git has did is, git is first appending the type of the content, which is blob in our case, we have seen hyphen T written our blob, then the total number of characters. So bytes, basically bytes for the content, and then the null terminating character that is after 14 bytes that content is basically ending and then the actual content.

This is how git is generating your hash code for the blob type. Is this now okay for everyone? How git is generating your hash code for you? Yes. Great, great.

Blob size batch 0 content. Yes, that's the format. I didn't get your question, Rahul.

If Shasm is converting the blob into the hash code, what is the code? So internally, git is creating your hash based on this, right? So whatever content you are passing, it is appending this part to the content and then whatever code is generated is basically is what you are getting it, right? So let's get going. Now, what we're going to do is, we'll first understand this hyphen w1, which I haven't been able to complete last time. So this hyphen w, okay.

But before that, do let me know if you can able to see this red, green and blue markings. Are you able to see that guys? Great. Okay.

So then you know what I'm going to do, I'm going to create memory layout, how these commands get executed. So let's see how this command gets executed in the RAM. We have operating system and then we have git.

First, let's say this is a bash and in bash we are writing echo command and echo hello world will come over here. I will use short form of hello world, right? And then we have git command, git program. This will be a git.

Git will receive this input. We already talked about this, so I will not spend time on it. Right, git hash object.

So git hash object will have output. I will use the starting two letters at AB, you know the entire key. This is what we will have output.

But at the same time, there are a few more things happening. So this is, imagine this is your hard disk. It has to be outside RAM, right? This is a volatile memory.

This is a persistent memory, right? Persistent storage. So let's say this is your hard disk. In your hard disk, you have dot git slash objects folder, right? Objects folder.

And in that objects folder, you have 8A as a folder. 8A and then remaining key, right? That is, this is a call going to happen. That's the one thing and very important.

The second thing would be your actual content. Your actual content would be written somewhere else. So this is just a file name data is writing and then your actual content which is hello world is

also written somewhere, right? I will prove that, okay? So just trust me.

I hope you guys trust me so far, right? So I will gonna prove anyway because I like to do things hands-on. So we'll prove that. So in hard disk, it is gonna return in git slash objects slash 8A.

So first letters are the folder of the key 8A and then B and something else. So from B to 6D, A6D till that we will have file, okay? All good. So this is how it writes into the hard disk.

And next time when you do git cat file, it basically fetches this key and that's the reason on the day one when we didn't write into hard disk, we didn't get key, right? And Siddharth had answered about state management. Clear so far guys? Yes. Are we good? How it is executing? Yes.

Great.

This file is longer than 30 minutes.

Go Unlimited at https://turboscribe.ai/ to transcribe files up to 10 hours long.